Machine Problem 4: Kernel-Level Thread Scheduling

Nithin Gowtham Saravanan
UIN: 337001954
CSCE611: Operating Systems

Assigned Tasks

Main: Completed.
Bonus Option 1: Correct handling of interrupts – Completed.
Bonus Option 2: Round-Robin Scheduling – Completed.
Bonus Option 3: Processes - Not applicable.
Bonus Option 4: Not applicable.

System Design

The objective of this machine problem is to design and implement **kernel-level threading and scheduling** with interrupt-driven preemption.
This work extends the existing base kernel with the following capabilities:

1. Kernel threads with their own stacks and register contexts.
2. A unified scheduler supporting both **FIFO (non-preemptive)** and **Round-Robin (RR)** (preemptive) scheduling.
3. Interrupt-driven context switching using the programmable interval timer (PIT).
4. Graceful thread termination, zombie reaping, and compatibility with the existing build system.

Scheduling Architecture

The scheduling design follows a modular and extensible structure:

- **Thread** objects represent execution contexts, each with a private stack and saved CPU state.
- The **Scheduler** manages runnable and zombie queues.
- The **Interrupt subsystem** dispatches hardware IRQs to handlers, including the timer ISR.
- **Timer** drivers (SimpleTimer / EOQTimer) generate periodic interrupts that drive preemption.
- The **Kernel** module wires these components together based on compile-time flags.

Key design requirement: the same kernel binary must operate in both cooperative (FIFO) and preemptive (RR) modes, selectable through #define flags

These are defined in macros_config.H

```
#define _USES_SCHEDULER_        // uncomment to enable FIFO scheduling
#define _TERMINATING_FUNCTIONS_ // uncomment to terminate threads 1 and 2 in kernel.c
#define _RR_SCHEDULER_          // uncomment to enable Round-Robin
```

Method of Implementation

1. **Thread bootstrap & context:** Each Thread is created with a fabricated interrupt frame so execution begins at thread_start(), which immediately enables interrupts (IF=1) and returns into the real thread function already pushed on the stack. When the function returns, thread_shutdown() takes over to terminate safely instead of "falling off" the stack.

2. **Low-level dispatch:** Thread::dispatch_to() calls the assembly switch routine (threads_low_switch_to) to save the old ESP, load the new one, and restore registers/segments. This path updates current_thread atomically and doesn't return until a later switch brings the caller back.

3. **Ready & zombie queues:** A simple FIFO list (head/tail) tracks runnable threads without peeking into private Thread fields. A separate LIFO list holds "zombie" threads (finished but not yet freed) so no one frees their own stack mid-execution; both lists are touched with interrupts disabled to avoid races with the timer.

4. **Yield & resume primitives:** yield() is cooperative: if the ready queue is empty, keep running; otherwise pop head and switch. add()/resume() push to the tail, and both perform opportunistic zombie cleanup to keep memory pressure low without dedicated reaper passes.

5. **Termination paths:** With _TERMINATING_FUNCTIONS_, a thread that returns (or calls terminate_self()) removes itself from the ready queue, becomes a zombie, and immediately switches to the next eligible thread (or last yielder/idle). External terminate(t) removes a non-running thread from the queue and reclaims its stack and TCB right away.

6. **Idle thread:** A tiny, never-enqueued idle thread exists as a safe sink when the ready queue is empty. It keeps interrupts enabled and just loops, ensuring the PIT keeps firing and the system doesn't deadlock.

7. **Interrupt dispatcher:** InterruptHandler::dispatch_interrupt() finds the handler and manages End-of-Interrupt (EOI) to the PIC. In RR builds, IRQ0 (timer) sends EOI before

calling the handler so the scheduler can context-switch inside the ISR without blocking the timer line; all other IRQs (and all FIFO builds) use the normal "handler then EOI" flow.

8. **Timers:** SimpleTimer programs the PIT divisor and counts ticks/seconds—handy as a heartbeat and a correctness check (you see "One second has passed"). EOQTimer derives from it and overrides handle_interrupt() to notify the scheduler every quantum, reusing the same PIT setup but changing what happens on each tick.
9. **RR preemption hook:** In RR mode, Scheduler::eoq_tick_isr() is an ISR-safe handoff that queues the running thread at the tail and dispatches the next runnable thread right from the interrupt context. This achieves time slicing without relying on cooperative yields and carefully avoids double-EOI or post-ISR surprises.
10. **Kernel wiring & build flags:** _USES_SCHEDULER_ enables the scheduler path; _RR_SCHEDULER_ switches from FIFO to RR and swaps SimpleTimer for EOQTimer; _TERMINATING_FUNCTIONS_ toggles returning thread functions. Kernel wiring only changes in a few #ifdef blocks, so the same source builds clean FIFO or RR systems—backward compatible and no linker/makefile tweaks.

Code Description

This implementation modifies the following files:

- thread.H
- thread.C
- scheduler.H
- scheduler.C
- simple_timer.H
- simple_timer.C
- interrupts.C
- macros_config.H
- kernel.C

thread.H

- **Safe resource cleanup (zombie reaping):** The scheduler needs limited access to each thread's stack base and size to properly free memory after a thread terminates. The get_stack_base() and get_stack_size() accessors expose only what's necessary—avoiding unsafe direct access to private members.
- **Controlled thread lifecycle management:** thread_start() acts as a uniform entry stub for all threads, setting up interrupt flags and calling the actual user function,

while thread_shutdown() is automatically invoked when the thread function returns—ensuring it doesn't "fall off" the stack but transitions cleanly to the scheduler.

- **Integration with termination support (_TERMINATING_FUNCTIONS_): These** hooks allow the scheduler to reclaim thread resources (stack + TCB) and manage zombies automatically, enabling both self-termination and externally triggered termination without memory corruption or dangling stacks.

```
    /* Accessors used by Scheduler for safe cleanup (zombie reaper) */
    /* These provide minimal, controlled access to stack base/size so the scheduler
       can free resources when reaping zombies. */
    char * get_stack_base() { return stack; }
    unsigned int get_stack_size() { return stack_size; }
};

/*--------------------------------------------------------------------------*/
/* SUPPORT FUNCTIONS */
/*--------------------------------------------------------------------------*/

/* Called automatically when a thread function returns.
   The scheduler uses it to safely terminate and context-switch away. */
extern "C" void thread_shutdown();

/* Entry stub for new threads */
extern "C" void thread_start();

#endif
```

thread.C

1) thread_start()

- Initializes the execution environment for a new thread — enables interrupts (IF flag) and then jumps to the user-defined thread function whose address was preloaded on the stack during thread creation.
- When the user function finishes execution, control automatically transfers to thread_shutdown() instead of returning to invalid memory, ensuring a clean exit path.

```
extern "C" void thread_start() {
    /* This function is used to release the thread for execution in the ready queue. */

    /* We need to add code, but it is probably nothing more than enabling interrupts. */
    Machine::enable_interrupts();      /* allow timer/IRQs once running */
    return;                            /* 'ret' to thread function (pre-pushed) */
}
```

2) **thread_shutdown()**
   - Handles safe thread termination by marking the current thread as finished, moving it to the zombie list (if enabled), and invoking the scheduler to switch to the next runnable thread.
   - Prevents a terminated thread from resuming execution or corrupting the kernel stack, ensuring consistent cleanup and stable context-switch behavior.

```
extern "C" void thread_shutdown() {
    /* This function should be called when the thread returns from the thread function.
       It terminates the thread by releasing memory and any other resources held by the thread.
       This is a bit complicated because the thread termination interacts with the scheduler.
     */

    #ifdef _USES_SCHEDULER_
        SYSTEM_SCHEDULER->terminate_self(); // switches away; never returns
        /* not reached */
    #else
        /* No scheduler */
        for(;;)
        {
            /* spin forever to avoid falling off the stack */
        }
    #endif
}
```

Note: extern "C" ensures these functions have unmangled names so they can be correctly linked and called from assembly code during thread setup and context switching.

scheduler.H

Function prototype declarations for

1. **terminate_self()** allows the scheduler to safely remove the currently running thread (used by thread_shutdown()) and trigger a context switch without corrupting the scheduler state.
2. **eoq_preempt_isr()** handles timer-based preemption in Round-Robin mode, invoked directly from the timer interrupt to end the current thread's quantum and schedule the next runnable thread.

```
    void terminate_self();
    /* helper used by thread_shutdown() for current thread */

    #ifdef _RR_SCHEDULER_
        /* Called from the timer IRQ when quantum expires (IRQ context). */
        void eoq_preempt_isr();
    #endif
```

scheduler.C

1. **idle_body()** - Infinite loop for the idle thread that simply keeps interrupts enabled. It never enqueues/yields; it's the last-resort runnable when nothing else is ready.

```
static void idle_body()
{
  for (;;)
  {
    // keep interrupts enabled so timer/IRQs work. no enqueue/yield here
    Machine::enable_interrupts();
  }
}
```

2. **rq_push_tail(Thread* t)** - Allocates a ReadyNode and appends it to the tail of the FIFO ready queue. Keeps rq_head/rq_tail consistent for O(1) enqueue.

```
/* enqueue helper (tail). */
static inline void rq_push_tail(Thread* t) {
  ReadyNode* n = new ReadyNode(t);
  if (!rq_tail)
    rq_head = rq_tail = n;
  else
  {
    rq_tail->next = n;
    rq_tail = n;
  }
}
```

3. **rq_pop_head()** - Removes the head node from the ready queue and returns its thread. Updates both head and tail, handling the empty-queue case.

```
/* dequeue helper (head). */
static inline Thread* rq_pop_head()
{
  if (!rq_head)
    return nullptr;
  //Console::puts("[sched] pop head T"); Console::puti(rq_head->th->ThreadId()); Console::puts("\n"); //debug
  ReadyNode* n = rq_head;
  rq_head = rq_head->next;
  if (!rq_head)
    rq_tail = nullptr;
  Thread* t = n->th;
  delete n;
  return t;
}
```

4. **rq_remove(Thread* target)** - Linear search to unlink a specific thread from the ready queue, fixing head/tail as needed. Returns whether the thread was found.

```
/* remove a specific thread from the ready queue (if present). */
static inline bool rq_remove(Thread* target) {
  ReadyNode* prev = nullptr;
  ReadyNode* cur  = rq_head;
  while (cur)
  {
    if (cur->th == target)
    {
      if (prev)
        prev->next = cur->next;
      else
        rq_head    = cur->next;
      if (rq_tail == cur)
        rq_tail = prev;
      delete cur;
      return true;
    }
    prev = cur;
    cur  = cur->next;
  }
  return false;
}
```

5. **enqueue_zombie(Thread* t)** - Pushes a terminated thread onto the zombie list for out-of-context cleanup. Keeps resource reclamation out of the terminating thread's context.

```
/* enqueue thread to zombie list (caller must hold interrupts disabled if required) */
static inline void enqueue_zombie(Thread* t) {
  if (!t)
    return;
  ZombieNode* n = new ZombieNode(t);
  n->next = zombie_head;
  zombie_head = n;
}
```

6. **cleanup_zombies()** - Walks the zombie list, frees each thread's stack via the TCB accessors, then deletes the TCB and node. Empties the list in one sweep.

```
/* cleanup: free stacks and delete Thread objects */
static inline void cleanup_zombies() {
  ZombieNode* cur = zombie_head;
  while (cur)
  {
    ZombieNode* nxt = cur->next;
    Thread* t = cur->th;
    /* free stack memory allocated by kernel (new char[]) */
    char* base = t->get_stack_base();
    if (base)
      delete[] base;
    delete t;
    delete cur;
    cur = nxt;
  }
  zombie_head = nullptr;
}
```

7. **Scheduler::Scheduler()** - Creates the idle_thread with a tiny private stack if it doesn't exist and prints a banner. Ensures there's always a runnable fallback.

```
Scheduler::Scheduler()
{
  if (!idle_thread)
  {
    char* idle_stack = new char[1024];
    idle_thread = new Thread(&idle_body, idle_stack, 1024);
  }
  Console::puts("Constructed Scheduler.\n");
}
```

8. **Scheduler::yield()** - Saves the current thread as last_yielder, cleans up zombies with interrupts disabled, and if the ready queue is non-empty pops the next thread. Dispatches to that thread; otherwise just returns and continues running.

```
void Scheduler::yield()
{
  Thread* me_dbg = Thread::CurrentThread();
  assert(me_dbg != nullptr); /* sanity check we have a running thread */

  last_yielder = me_dbg;            /* remember who yielded */

  /* cooperative FIFO — current was already enqueued by pass_on_CPU()->resume() */
  Machine::disable_interrupts();    /* guard queue ops against timer IRQs */

  cleanup_zombies();                /* ensures no leak when only yielding */

  if (!rq_head) {                   /* nobody ready → keep running */
    Machine::enable_interrupts();
    return;
  }

  Thread* next = rq_pop_head();  /* pick next from head */
  Machine::enable_interrupts();  /* leave critical section */

  Thread::dispatch_to(next);     /* context switch; returns when we run again */
}
```

9. **Scheduler::resume(Thread* t)** - With interrupts disabled, reaps zombies and enqueues t at the tail (ignoring the idle thread). This is used for making a blocked/preempted thread runnable again.

```
void Scheduler::resume(Thread * _thread)
{
  /* Add the given thread to the ready queue of the scheduler. This is called
     for threads that were waiting for an event to happen, or that have
     to give up the CPU in response to a preemption. */
  if (!_thread)
   return;
  Machine::disable_interrupts();                /* guard queue ops */

  /* reap any zombies (runs while interrupts disabled) */
  cleanup_zombies();

  if (_thread == idle_thread)    /* never enqueue idle */
  {
    Machine::enable_interrupts();
    return;
  }
//   Console::puts("[sched] resume T");            /* trace */
//   Console::puti(_thread->ThreadId());
//   Console::puts("\n");
  rq_push_tail(_thread);                        /* enqueue at tail */
  Machine::enable_interrupts();                 /* done */
}
```

10. **Scheduler::add(Thread* t)** - Similar to resume() but intended for first-time runnable threads right after creation. It also opportunistically reaps zombies and never enqueues the idle thread.

```
void Scheduler::add(Thread * _thread)
{
  /* Make the given thread runnable by the scheduler. This function is called
     after thread creation. Depending on implementation, this function may
     just add the thread to the ready queue, using 'resume'. */
  if (!_thread)
    return;
  Machine::disable_interrupts();                /* guard queue ops */

  /* opportunistically reap any zombies */
  cleanup_zombies();

  if (_thread == idle_thread)       /* never enqueue idle */
  {
    Machine::enable_interrupts();
    return;
  }
  // Console::puts("[sched] add T");               /* trace */
  // Console::puti(_thread->ThreadId());
  // Console::puts("\n");
  rq_push_tail(_thread);                        /* enqueue at tail */
  Machine::enable_interrupts();                 /* done */
}
```

11. **Scheduler::terminate(Thread* t)** - If t is the current thread, it removes it from the RQ, parks it as a zombie (not the idle), selects a successor (head of RQ → last_yielder → idle), and dispatches—never returning. If t is not current, it removes it from the RQ if present, then immediately frees its stack and deletes the TCB.

```
void Scheduler::terminate(Thread * _thread)
{
  /* Remove the given thread from the scheduler in preparation for
destruction
     of the thread.
     Graciously handle the case where the thread wants to terminate
itself.*/
  if (!_thread)
    return;

  Thread* me = Thread::CurrentThread();

  if (_thread == me) {                          /* running thread
wants to terminate */
    Machine::disable_interrupts();              /* atomic pick of
successor */

    /* don't ever reap/queue idle thread as zombie */
    if (me != idle_thread)  /* protect idle thread from deletion */
    {
      rq_remove(me);
      enqueue_zombie(me);
    }
    else
    {
      rq_remove(me);        /* keep idle out of RQ if it ever gets
here */
    }

    Thread* next = rq_pop_head();               /* choose next
runnable (if any) */
    Machine::enable_interrupts();

    if (!next)
    {
      if (last_yielder)
      {
        Machine::disable_interrupts();
        rq_remove(last_yielder);
        Machine::enable_interrupts();
```

```
        // Console::puts("[sched] terminate: switching to
last_yielder T");
        // Console::puti(last_yielder->ThreadId());
Console::puts("\n");
        Thread::dispatch_to(last_yielder);
        return;
      }
      next = idle_thread;   // fallback (existing)
    }

    /* DO NOT free your own stack here; another thread must reap it
safely. */

    Thread::dispatch_to(next);                  /* never returns */
    return;                                     /* not reached */
  }

  /* terminating a non-running thread: best-effort remove from ready
queue. */
  if (_thread == idle_thread)     /* never delete idle explicitly */
  {
    return;
  }

  Machine::disable_interrupts();                /* guard queue ops
*/
  bool removed = rq_remove(_thread);            /* remove if
enqueued */
  (void)removed;
  Machine::enable_interrupts();

  /* Reclaim resources now that we are not the thread itself. */
  char* base = _thread->get_stack_base();
  if (base) delete[] base;
  delete _thread;
}
```

12. **Scheduler::terminate_self()** - Called by thread_shutdown() on return from a thread function; it removes the current thread from the RQ, parks it as a zombie (unless idle), and picks the next thread exactly like the self-path in terminate(). Dispatches to the successor and doesn't return.

```
• /* helper used by thread_shutdown() for current thread */
• void Scheduler::terminate_self()
• {
```

```
  // Console::puts("[sched] terminate_self ENTER\n");
  // Console::puts("[sched] TERMINATE_SELF PATCH ACTIVE\n");
  Machine::disable_interrupts();

  /* reap zombies here as well to eliminate leaks when only terminations happen */
  cleanup_zombies();    /* reaper on self-termination path */

  Thread* me = Thread::CurrentThread();

  /* remove ourselves from ready queue if present */
  rq_remove(me);

  /* park current thread as zombie (reaped later), unless it's the idle thread */
  if (me != idle_thread) /* don't zombie the idle thread */
  {
    enqueue_zombie(me);
  }

  Thread* next = rq_pop_head();
  // Console::puts(next ? "[sched] terminate_self: have next\n"
  //                    : "[sched] terminate_self: rq empty\n");
  Machine::enable_interrupts();

  if (!next)
  {
    //Console::puts("[sched] terminate_self: rq empty");
Console::puts("\n");  /* trace */

    if (last_yielder)   /* prefer the last thread that yielded */
    {
      Machine::disable_interrupts();
      rq_remove(last_yielder);          /* avoid duplicate if it was enqueued */
      Machine::enable_interrupts();
      // Console::puts("[sched] terminate_self: switching to last_yielder T");
      // Console::puti(last_yielder->ThreadId());
Console::puts("\n");
      Thread::dispatch_to(last_yielder);  /* never returns */
      return;
    }
```

```
•
•       //Console::puts("[sched] terminate_self: switching to
    idle\n");            /* keep idle as last resort */
•       next = idle_thread;
•     }
•     else
•     {
•       // Console::puts("[sched] terminate_self: next = ");
•       // Console::puti(next->ThreadId());
•       // Console::puts("\n");
•     }
•
•     Thread::dispatch_to(next);
•   }
```

Note: last_yielder (state usage) - Tracks the most recent thread that voluntarily called yield() to provide a fair fallback when the ready queue is empty during termination paths. Helps avoid getting stuck on the idle thread if a productive

13. **eoq_preempt_isr()** *(RR builds only)* - Runs in IRQ context on end-of-quantum, moving the current thread to the ready queue and selecting the next runnable. It arranges the context switch safely from inside the timer interrupt so preemption feels transparent to the threads.

```
#ifdef _RR_SCHEDULER_
void Scheduler::eoq_preempt_isr() {
  /* IRQ context: interrupts are already disabled by CPU.
     Do not call Machine::{enable,disable}_interrupts() here.
     Do not print to console. Keep it minimal & reentrant. */

  Thread* me = Thread::CurrentThread();
  if (!me || me == idle_thread) {
    return;                        /* nothing to preempt */
  }

  /* Put current at tail; pick next from head.  RQ helpers are file-local
     and not touching Thread internals; safe with IRQs disabled. */
  rq_push_tail(me);
  Thread* next = rq_pop_head();

  if (!next || next == me) {
    return;                        /* nobody else runnable */
  }

  /* Context-switch INSIDE the IRQ handler. We've arranged early EOI (see interrupts.C),
     so it's safe that we won't "return" to send EOI after the handler. */
  Thread::dispatch_to(next);
  /* never returns here; when the preempted thread is later rescheduled,
     it will resume after the dispatcher in the ISR frame and return normally. */
}
#endif
```

simple_timer.H

The EOQTimer class extends SimpleTimer to provide periodic quantum-expiry interrupts for Round-Robin scheduling. It triggers Scheduler::eoq_preempt_isr() on each tick to preempt the running thread when its time slice ends. Registered on IRQ0 only in RR builds, it replaces the standard timer without requiring build or linker changes.

```
#ifdef _RR_SCHEDULER_
/*--------------------------------------------------------------------------*/
/* E O Q   T i m e r   (Round-robin quantum timer)                           */
/*--------------------------------------------------------------------------*/
/* A periodic timer used by the RR scheduler. On every tick, it informs the
   scheduler that the time slice has ended so the running thread may be
   preempted. This is installed on IRQ0 instead of SimpleTimer in RR builds. */
class EOQTimer : public SimpleTimer {
public:
  /* quantum_ms: time slice in milliseconds (e.g., 50 for 20Hz preemption). */
  explicit EOQTimer(int quantum_ms);
  virtual void handle_interrupt(REGS* _r);
};
#endif
```

simple_timer.C

The EOQTimer implementation in simple_timer.C extends the base SimpleTimer to support time-slice–based preemption for Round-Robin scheduling.

- **Constructor (EOQTimer(int quantum_ms)):** Initializes the timer by converting the quantum duration (in milliseconds) into an interrupt frequency for the Programmable Interval Timer (PIT). For example, a 50 ms quantum sets the timer to 20 Hz (1000 / 50 = 20 ticks per second).
- **handle_interrupt(REGS* _r):** Called on each timer interrupt (IRQ0). Instead of printing "tick" messages, it directly invokes Scheduler::eoq_preempt_isr() to signal that the running thread's time slice has ended, allowing preemption to occur safely inside the interrupt context.

```
#ifdef _RR_SCHEDULER_
#include "scheduler.H"   /* for SYSTEM_SCHEDULER declaration */
extern Scheduler* SYSTEM_SCHEDULER;

EOQTimer::EOQTimer(int quantum_ms)
  : SimpleTimer((quantum_ms > 0) ? (1000 / quantum_ms) : 20) {}

/* Delegate to the scheduler's IRQ-safe preemption hook. */
void EOQTimer::handle_interrupt(REGS* /*_r*/) {
#ifdef _USES_SCHEDULER_
```

```
  if (SYSTEM_SCHEDULER) {
    SYSTEM_SCHEDULER->eoq_preempt_isr();  /* IRQ context; no prints, no
(en|dis)able */
  }
#endif
}
#endif
```

interrupts.C

- Added conditional handling for IRQ0 (timer interrupts) when _RR_SCHEDULER_ is defined.
- Sent the End-of-Interrupt (EOI) signal before invoking the timer handler to avoid PIC deadlock during in-ISR context switches.
- Ensured the scheduler's preemption logic (eoq_preempt_isr) can safely execute without blocking future timer interrupts.
- Retained the original EOI-after-handler flow for FIFO scheduling and other IRQs, ensuring full backward compatibility.

```
if (!handler) {
  /* --- NO DEFAULT HANDLER HAS BEEN REGISTERED. SIMPLY RETURN AN ERROR. */
  Console::puts("INTERRUPT NO: ");
  Console::puti(int_no);
  Console::puts("\n");
  Console::puts("NO DEFAULT INTERRUPT HANDLER REGISTERED\n");
  //    abort();
}
else {
if defined(_USES_SCHEDULER_) && defined(_RR_SCHEDULER_)
  /* In RR mode, pre-ACK the timer IRQ (IRQ0) so the ISR can trigger a
     context switch without leaving the PIC line wedged. */
  if (int_no == 0) {
    if (generated_by_slave_PIC(int_no)) {
      Machine::outportb(0xA0, 0x20);
    }
    Machine::outportb(0x20, 0x20);
    handler->handle_interrupt(_r);
    return;  /* avoid double EOI below */
  }
endif
  /* FIFO build or non-timer IRQ in RR: handle first, then EOI. */
  handler->handle_interrupt(_r);
}
```

macros_config.H

Since the macros defined in kernel.C are used across multiple files, a new header file defining those macros is created and this file actually defines the values for the entire project. The #defines in kernel.C are made dummy and the code relies only on the values from this file.

```
#define _USES_SCHEDULER_        // uncomment to enable FIFO scheduling
#define _TERMINATING_FUNCTIONS_ // uncomment to terminate threads 1 and 2 in kernel.c
#define _RR_SCHEDULER_          // uncomment to enable Round-Robin
```

kernel.C

- When _RR_SCHEDULER_ is defined, it instantiates and registers the EOQTimer instead of the default SimpleTimer, enabling periodic preemption for Round-Robin scheduling.
- For FIFO or non-RR builds, the original SimpleTimer remains in use, maintaining compatibility with existing behavior and avoiding unnecessary timer overhead.

```
#ifndef _RR_SCHEDULER_
    // FIFO build - use the original SimpleTimer (e.g., 10ms for "one second passed" prints)
    SimpleTimer timer(100);            /* 10ms tick */
    InterruptHandler::register_handler(0, &timer);
    /* The Timer is implemented as an interrupt handler. */
#else
    EOQTimer eoq(50);                  /* 50ms RR quantum */
    InterruptHandler::register_handler(0, &eoq);
#endif
```

Bonus Option 1: Correct handling of interrupts

- **Re-enable interrupts when a thread first starts (thread.C):** New threads begin with IF=0 in EFLAGS (by design of the bootstrap context). We added Machine::enable_interrupts() in thread_start() so, after the fake "exception return," the CPU has interrupts enabled and the periodic timer ("One second has passed") resumes.
- **Guard all ready-queue mutations with CLI/STI (scheduler.C):** In yield(), resume(), add(), terminate(), and terminate_self() we wrap enqueue/dequeue/zombie-list operations with Machine::disable_interrupts() / Machine::enable_interrupts() to provide mutual exclusion against timer/IRQ handlers touching shared state.
- **Never enqueue the idle thread & clean up safely (scheduler.C):** We ensure the idle thread is never placed on the ready queue and we use a zombie-reaper path that runs

with interrupts disabled while it unlinks/frees finished threads, then re-enables interrupts promptly—preventing IRQ races during reclamation.

- **Leave context switches themselves interrupt-safe (thread/scheduler split):** Interrupts are disabled only for the minimal critical sections; we re-enable them before calling Thread::dispatch_to(next) so the switched-in thread can run with interrupts enabled unless it explicitly disables them.
- **Kernel integration unchanged except timer now ticks post-start (kernel.C):** With thread_start() enabling interrupts, the already-installed SimpleTimer on IRQ0 fires normally during thread execution—demonstrating that global interrupt state is now managed correctly by thread management and the scheduler.

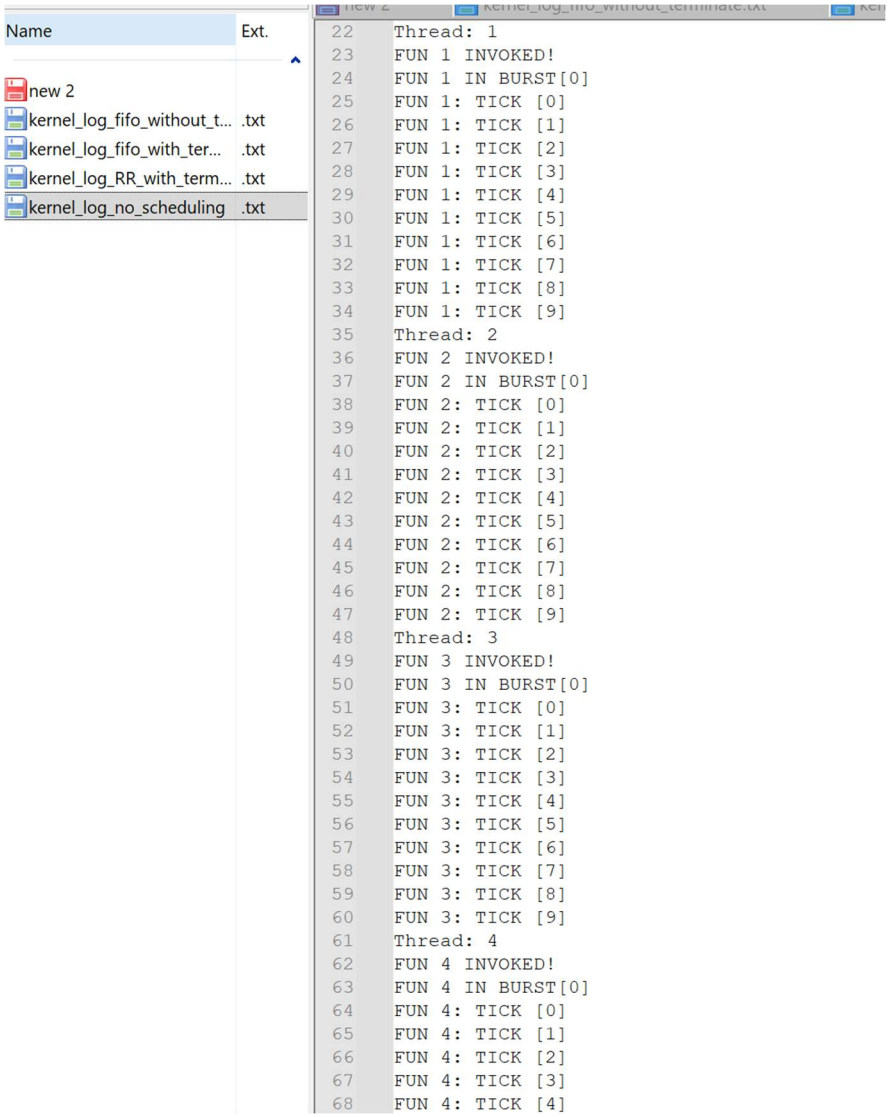
Bonus Option 2: Round-Robin Scheduling

- **EOQTimer derived from SimpleTimer (simple_timer.H/.C):** Added class EOQTimer : public SimpleTimer. The constructor converts the quantum (ms) to PIT frequency (hz = 1000/quantum_ms, default ~20 Hz). Its ISR simply calls SYSTEM_SCHEDULER->eoq_preempt_isr(), keeping all policy in the scheduler.
- **Kernel timer selection (kernel.C):** With _RR_SCHEDULER_ defined, IRQ0 uses EOQTimer (e.g., 50 ms quantum); otherwise, it installs the standard SimpleTimer. Ensures backward-compatible FIFO behavior.
- **Scheduler preemption (scheduler.H/.C):** Implemented eoq_preempt_isr() for IRQ-safe preemption. On each tick, if the running thread is runnable, it is marked READY, re-queued, and the next thread is dispatched, providing time-sliced rotation over the FIFO queue.
- **Safe ISR context switching (interrupts.C):** For IRQ0 in RR builds, the dispatcher sends EOI to the PIC before invoking the timer handler, allowing safe stack switching inside the ISR and avoiding duplicate EOIs.
- **Minimal critical section (scheduler.C):** Uses short CLI-protected sections for queue updates; no prints or blocking calls inside the ISR. Context switch via Thread::dispatch_to(next) remains fully threads_low.asm–compatible.
- **Idle & zombie handling (scheduler.C):** The idle thread is never preempted; finished threads are moved to the zombie list instead of being re-enqueued, maintaining correct round-robin rotation.
- **Compatibility:** No makefile or assembly changes. FIFO + SimpleTimer remains default; defining _RR_SCHEDULER_ enables EOQTimer-based time slicing, EOI-before-handler logic, and full interrupt-driven preemption.

TESTING

A. No scheduling and no termination

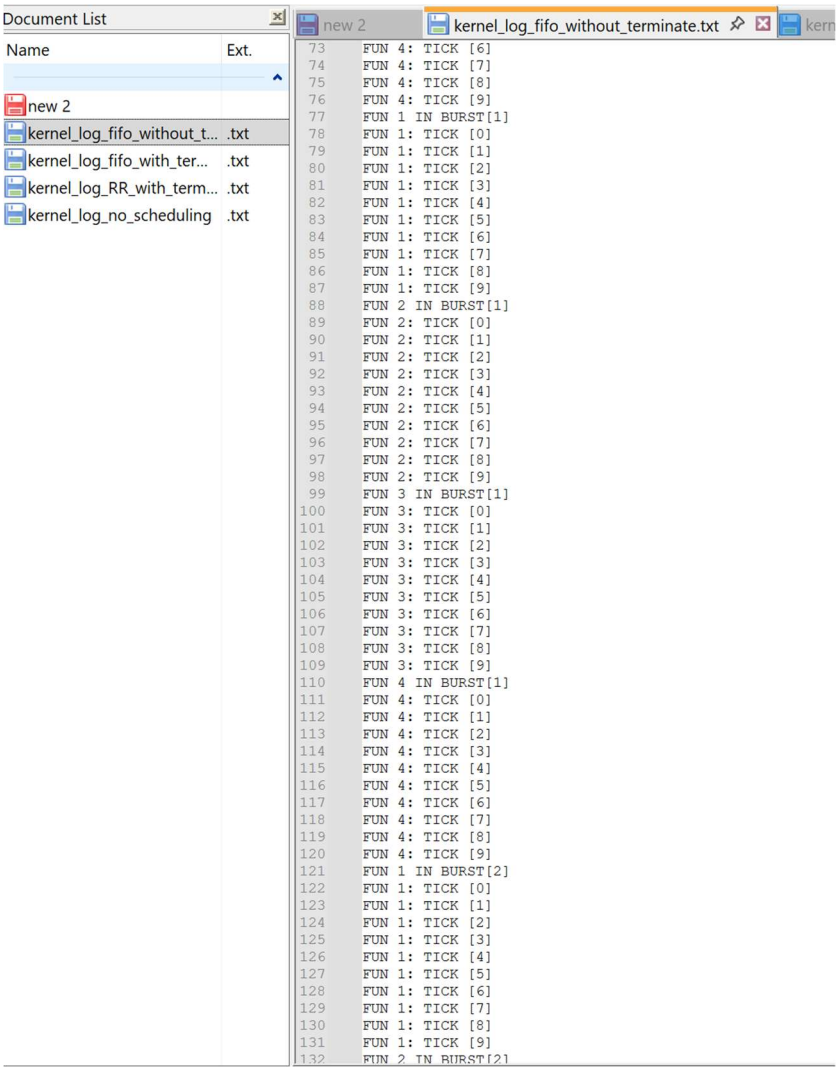
- Setup: Scheduler disabled; plain SimpleTimer on IRQ0; threads run to completion without termination logic. All the macros in macros_config.H are disabled.
- After “Installed interrupt handler at IRQ <0> / Hello World!”, each thread runs its full 10-tick “burst” before the next starts—strict sequential execution with no preemption or ready queue rotation.
- Checks passed: No context switches mid-burst; eventually you saw “One second has passed,” showing the PIT interrupt still fires (but it doesn’t drive scheduling). No zombie handling is exercised here.



A detailed log is stored at **kernel_log_no_scheduling.txt**. This is executed with the command **make run > kernel_log_no_scheduling.txt 2>&1**

B. Basic FIFO scheduling and no termination

- Setup: Scheduler enabled (note "Constructed Scheduler."), non-terminating thread functions; still using SimpleTimer (not EOQTimer); interrupts re-enabled per Bonus 1 fix. Enable the macro **"_USES_SCHEDULER_"** in macros_config.H
- Thread creation prints, then rotation in FIFO order: T1 burst(0) → T2 burst(0) → T3 → T4, then burst(1) for each, and so on. Each gets the CPU and runs its whole burst (10 ticks) before yielding.
- Checks passed: Correct FIFO discipline (enqueue at tail, dequeue at head) with no mid-burst preemption. Timer messages ("One second has passed") appear over long runs, proving interrupts were properly re-enabled after the first thread started (matching Bonus 1 requirements).

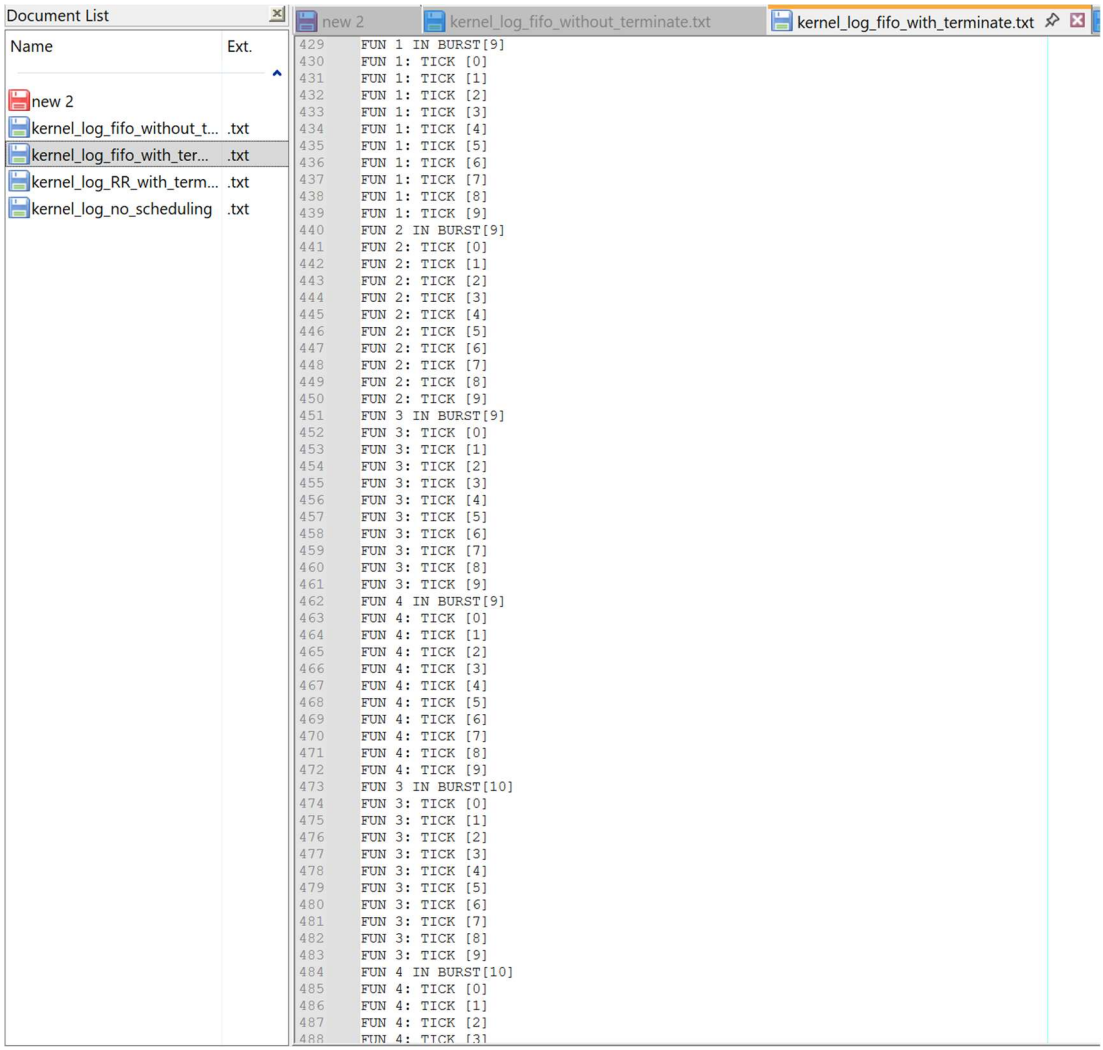


A detailed log is stored at **kernel_log_fifo_without_terminate.txt**. This is executed with the command

make run > kernel_log_fifo_without_terminate.txt 2>&1

C. Basic FIFO scheduling with termination

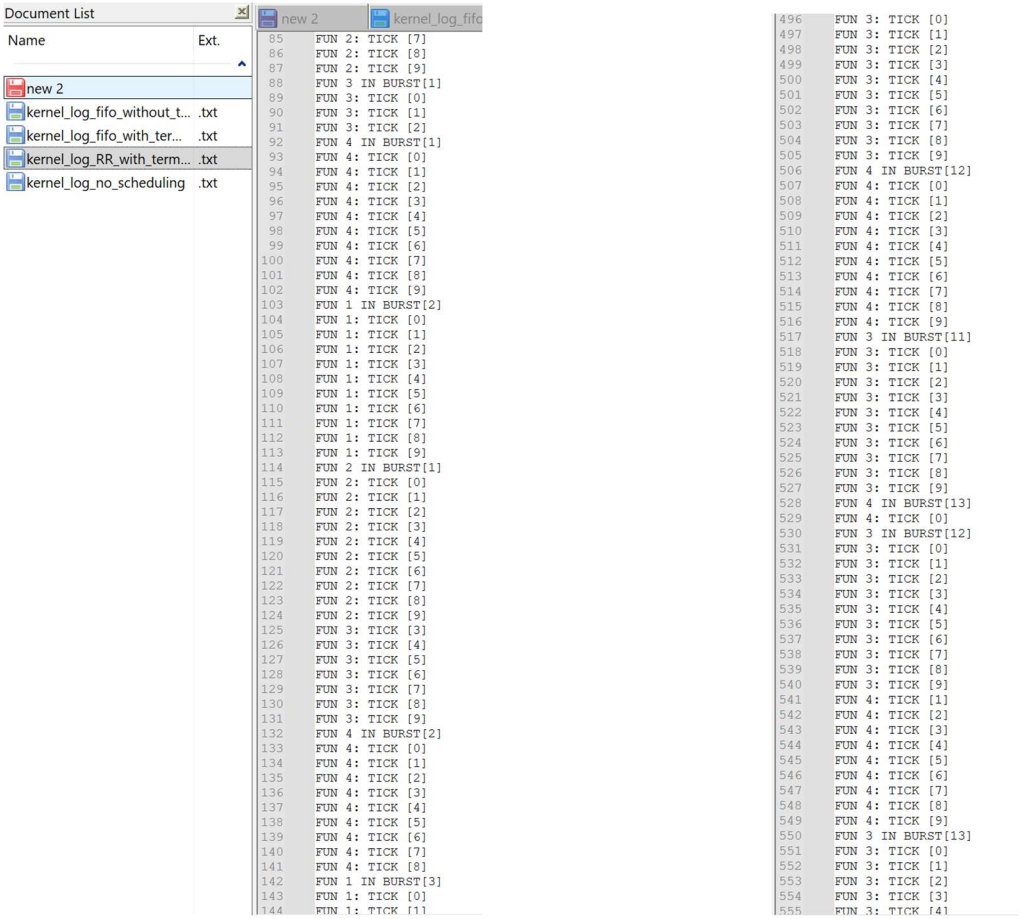
- Setup: Same FIFO scheduler, but thread functions do finish; zombie reaping enabled.
- Enable the macros "_USES_SCHEDULER_" and "_TERMINATING_FUNCTIONS_" in macros_config.H
- Normal FIFO rotation at first; when a thread completes, it stops appearing in later bursts. Rotation continues among remaining threads without gaps or repeats.
- Checks passed: Finished threads are not rescheduled (moved to zombie list and reaped); ready queue shrinks cleanly; no "stuck" or resurrected threads—shows safe hand-off from thread_shutdown() to the scheduler's terminate/reap path.
- Thread 1 and 2 terminate at burst[9] and only thread 3 and 4 continues.



A detailed log is stored at **kernel_log_fifo_with_terminate.txt**. This is executed with the command **make run > kernel_log_fifo_with_terminate.txt 2>&1**

D. Round-Robin scheduling with termination

- Setup: _RR_SCHEDULER_ build: EOQTimer installed on IRQ0 (e.g., 50 ms quantum), EOI-before-handler in interrupts.C, RR preemption implemented in eoq_preempt_isr().
- Enable all the macros "_USES_SCHEDULER_", "_TERMINATING_FUNCTIONS_" and "_RR_SCHEDULER_" in macros_config.H
- Mid-burst preemption: e.g., a thread prints ticks [0..7], then another thread runs, later the first resumes at [8][9]. This interleaving repeats, and terminating threads drop out of rotation as they finish.
- Checks passed: Time-sliced rotation across runnable threads (not just at burst boundaries), clean removal of finished threads, and no double-EOI or wedged timer (proved by stable continued interleaving and lack of spurious returns from old ISRs). Backwards compatibility also holds because FIFO builds keep SimpleTimer and never call RR paths. It is also observed that more than a single thread runs in a particular burst ensuring that RR scheduling is implemented correctly. **This ensures that bonus options 1 and 2 are satisfied. Also thread 1 and 2 are terminated after 10 bursts.**



A detailed log is stored at **kernel_log_RR_with_terminate.txt**. This is executed with the command **make run > kernel_log_RR_with_terminate.txt 2>&1**